

give a summary table of search algorithms, with their complexities and best uses

Here's a **clean, interview-friendly summary table of common search algorithms**, their **time/space complexities**, and **when you should use them**. I've grouped them so the patterns are obvious.

1. Array / List Search Algorithms

Algorithm	Data Requirement	Time Complexity	Space	Best Use
Linear Search	Unsorted	$O(n)$	$O(1)$	Small datasets, unsorted data, one-off searches
Binary Search	Sorted	$O(\log n)$	$O(1)$ (iterative)	Fast lookups on sorted arrays
Jump Search	Sorted	$O(\sqrt{n})$	$O(1)$	When random access is cheap, fewer comparisons than linear
Interpolation Search	Sorted + uniform distribution	$O(\log \log n)$ avg, $O(n)$ worst	$O(1)$	Numeric data with near-uniform distribution

2. Hash-Based Search

Algorithm	Data Requirement	Time Complexity	Space	Best Use
Hash Table Lookup	Hashable keys	$O(1)$ avg, $O(n)$ worst	$O(n)$	Fast key-based lookups (dicts, sets)

💡 *Worst case happens due to hash collisions.*

3. Tree-Based Search

Algorithm	Data Structure	Time Complexity	Space	Best Use
BST Search	Binary Search Tree	$O(\log n)$ avg, $O(n)$ worst	$O(h)$	Ordered data with dynamic inserts
Balanced BST (AVL, Red-Black)	Self-balancing tree	$O(\log n)$	$O(h)$	Guaranteed performance
Trie Search	Prefix tree	$O(L)$	High	Prefix / autocomplete / dictionary search

$L = \text{length of the key}$

4. Graph Search Algorithms

Algorithm	Traversal Type	Time Complexity	Space	Best Use
DFS	Depth-first	$O(V + E)$	$O(V)$	Explore paths, cycle detection
BFS	Breadth-first	$O(V + E)$	$O(V)$	Shortest path in unweighted graphs
Bidirectional BFS	BFS from both ends	$O(b^{(d/2)})$	$O(b^{(d/2)})$	Faster shortest path when start & end known

5. Heuristic / Informed Search (AI-style)

Algorithm	Uses Heuristic	Time Complexity	Space	Best Use
Greedy Best-First Search	Yes	$O(b^d)$	$O(b^d)$	Fast but not optimal
A*	Yes + cost	$O(b^d)$	$O(b^d)$	Optimal shortest path (admissible heuristic)
Dijkstra	No heuristic	$O((V + E) \log V)$	$O(V)$	Shortest paths, non-negative weights

6. Special Search Problems

Algorithm	Problem Type	Time Complexity	Best Use
Exponential Search	Sorted array	$O(\log n)$	When search space size is unknown
Fibonacci Search	Sorted array	$O(\log n)$	Cache-friendly binary search variant
Ternary Search	Unimodal function	$O(\log n)$	Optimization problems

Quick Interview Heuristics

- **Unsorted data?** → Linear Search
- **Sorted array?** → Binary Search
- **Fast key lookup?** → Hash Table
- **Prefix / autocomplete?** → Trie
- **Shortest path (no weights)?** → BFS
- **Shortest path (weights)?** → Dijkstra / A*
- **AI / pathfinding?** → A*

If you want, I can:

- Convert this into a **LeetCode-style decision chart**
- Add **Python code snippets**
- Or give **“which one would you pick and why?” interview questions**



explain the the full binary search process, intuition, pseudocode, and then algo

Intuition (what binary search is “doing”)

Binary search is a **repeated halving** strategy on a **sorted** array.

- You keep a search window: `low ... high`
- Look at the middle element `mid`
- If `arr[mid]` is:
 - **Equal** to target → done
 - **Too small** → target must be in the **right half** → move `low = mid + 1`
 - **Too large** → target must be in the **left half** → move `high = mid - 1`

Why it works: because the array is sorted, so **everything left of** `mid` **is $\leq$** `arr[mid]` and everything right is $\geq$ it. One comparison lets you discard half the candidates.

Full process (step-by-step mental model)

Start with:

1. `low = 0, high = n - 1`
2. While `low <= high`:
 - `mid = low + (high - low) // 2` (*safe from overflow in some languages*)
 - Compare `arr[mid]` with target
 - Shrink the window to the half that can still contain the target
3. If the loop ends, the target is not present.

Loop condition `low <= high` means: “there is still at least one element in the window.”

Pseudocode (language-agnostic)

kotlin

```
binary_search(arr, target):  
    low  = 0  
    high = length(arr) - 1  
  
    while low <= high:  
        mid = low + (high - low) // 2  
  
        if arr[mid] == target:  
            return mid  
        else if arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return -1
```

Algorithm in Python (clean + interview-friendly)

python

```
def binary_search(arr, target):  
    """  
    Returns the index of target in sorted arr, or -1 if not found.  
    Assumes arr is sorted in non-decreasing order.  
    """  
    low, high = 0, len(arr) - 1  
  
    while low <= high:  
        mid = low + (high - low) // 2  
  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return -1
```

```
        high = mid - 1

    return -1

def main():
    arr = [1, 3, 5, 7, 9, 11, 15]
    tests = [1, 7, 15, 2, 10, 16]

    for t in tests:
        idx = binary_search(arr, t)
        print(f"target={t}>2} -> index={idx}")

if __name__ == "__main__":
    main()
```

Key details interviewers like

- **Precondition:** array must be sorted
- **Time:** $O(\log n)$ comparisons
- **Space:** $O(1)$ (iterative)
- **The safe mid formula:** $low + (high - low) // 2$

 Copy code

If you want, I can also show the “**leftmost/rightmost occurrence**” variant (the one that trips people up), which is the real interview version of binary search.

